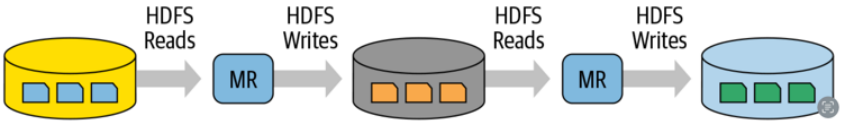
Introduction

History

- Big data at Google
- Distributed computing
- MapReduce → Apache Hadoop



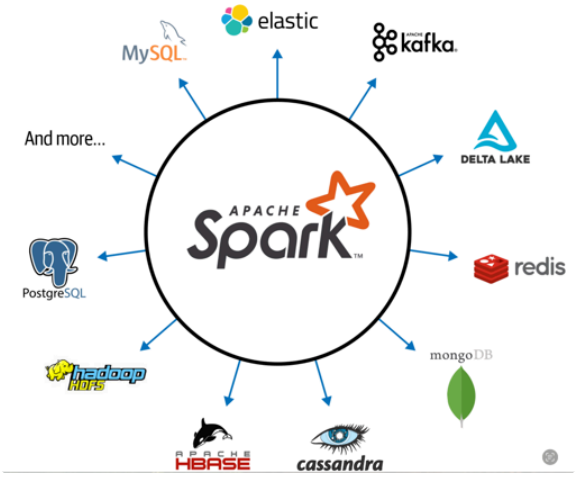
Apache Spark

> "Apache Spark is a unified engine designed for large-scale distributed data processing, on premises in data centers or in the cloud."

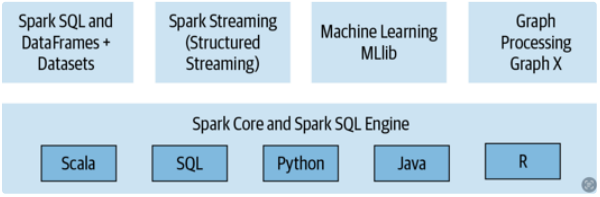
Excerpt From: Learning Spark, Jules S. Damji

- Donated by AMPLab at UC Berkley, Currently maintained by The Apache Foundation
- The group of founders start a company named Databricks

Spark connects to everything



Available modules and interfaces

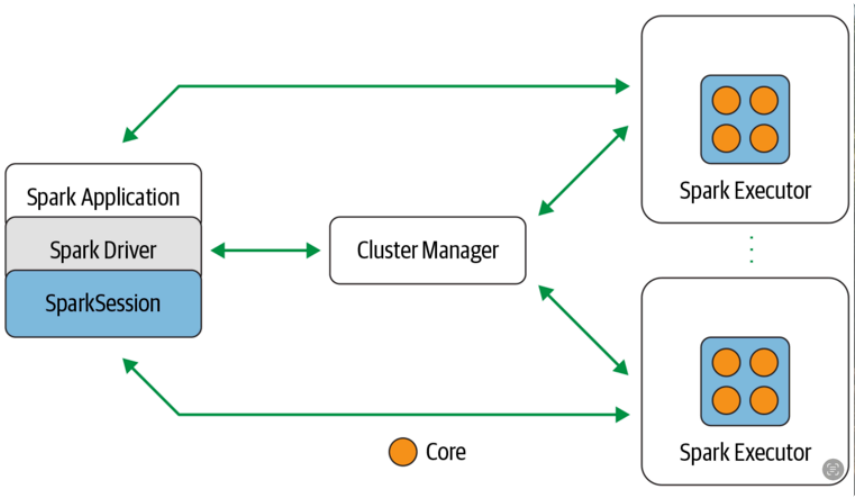


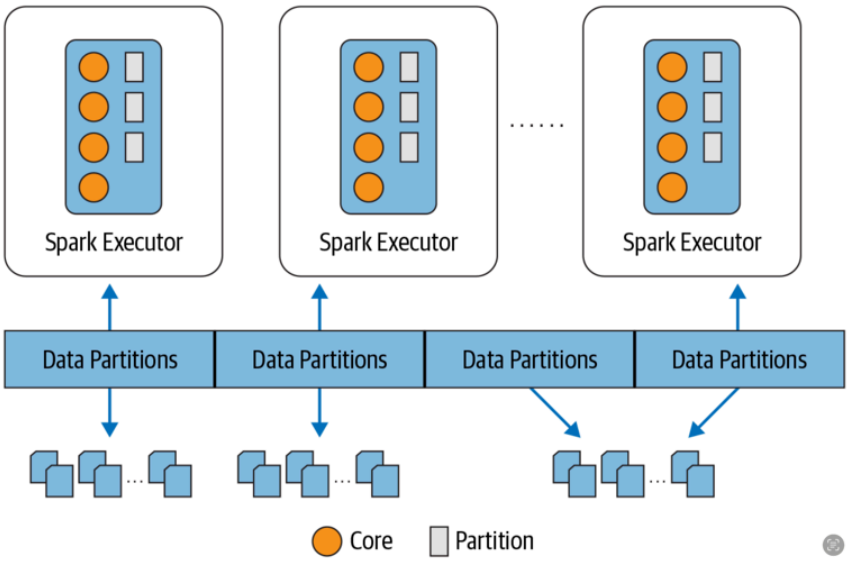
ML lib in Spark

```python
# In Python
from pyspark.ml.classification import LogisticRegression
...
training = spark.read.csv("s3://...")
test = spark.read.csv("s3://...")
# Load training data
lr = LogisticRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)
# Fit the model
lrModel = lr.fit(training)
# Predict
lrModel.transform(test)
```

Spark executes with lazy evaluation

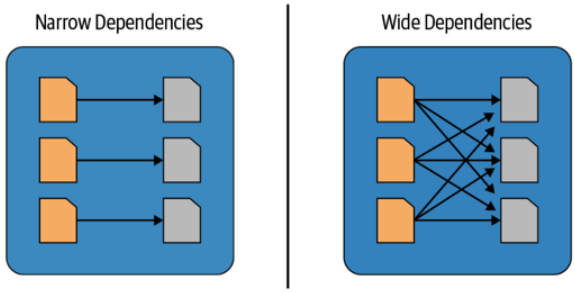
- Declarative language compatible - SQL
- Plan and execute





Narrow and wide dependencies

- Narrow, e.g.: `filter()`, `contains()`
- Wide, e.g.: `groupBy()`, `orderBy()`



File format

Apache Parquet, a columnar file format. Documentation

- Reading from CSV vs Parquet

Apache Spark vs Pandas

Feature	Pandas	Apache Spark
Location	Single Machine (RAM)	Cluster of Machines (Distributed)
When it runs	Immediately (Eager)	When an 'Action' is called (Lazy)
Scaling	Vertical (Buy a bigger RAM stick)	Horizontal (Add more computers)
Hardware	1 CPU Core	Many CPU Cores across network

Class Challenge: "The Case of the Crashing Candy Counter"

Scenario: You are a Data Engineer at a global confectionery company. Every factory sensor logs the color of every single M&M produced, generating a massive CSV file daily.

On **Day 1**, you receive a sample file (`mnm_sample.csv`) that is **500 MB**. You write the following Python script to count the colors, and it works perfectly in seconds:

```
# The Junior Engineer's Script
import pandas as pd

# Load data
df = pd.read_csv("mnm_data.csv")

# Count colors
counts = df.groupby("Color")["Count"].sum()
print(counts)

```

On **Day 2**, production goes into full swing. You receive the full production log (`mnm_full.csv`), which is **2 Terabytes**. You run the *exact same script* on the same machine (which has 32GB of RAM).

The Result: The script runs for 2 minutes, the fan on your laptop spins at maximum speed, and then the program crashes with a `MemoryError` .

The Questions for Discussion

1. **The Architecture Diagnosis:**

> "Explain exactly *why* the Pandas script crashed on Day 2 using the concept of **In-Memory Computing**. Why couldn't the computer just read the file in small chunks automatically?"

2. **The Spark Solution:**

> "If you rewrote this using `spark.read.csv(...)`, the code would look almost identical. However, Spark would likely complete the task successfully on a cluster.
>
> Explain how **partitions** allow Spark to process a 2TB file even if no single machine in the cluster has 2TB of RAM."

3. **How lazy evaluation might work here?:**

> Hint: `df.groupBy("Color").count()`